



Exercise: Rename to Redesign Recap

Python Programming Fundamentals

Naming as Design — Functions and Variables

Programming Fundamentals Team

SETU

2026-09-20



Outline

1. Today
2. One Function, One Job
3. Vague Names Hide Vague Purpose
4. Name the What, Not the How
5. Names Must Tell the Truth
6. Grammar Matches the Return
7. Flag Parameters Mean Two Functions
8. Long Names Are a Warning
9. Be Consistent
10. Naming Variables
11. Don't Shadow the Built-ins
12. Python Conventions (PEP 8)
13. Exercise: Rename to Redesign
14. Recap



Outline

1. Today

2. One Function, One Job

3. Vague Names Hide Vague Purpose

4. Name the What, Not the How

5. Names Must Tell the Truth

6. Grammar Matches the Return

7. Flag Parameters Mean Two Functions

8. Long Names Are a Warning

9. Be Consistent

10. Naming Variables

11. Don't Shadow the Built-ins

12. Python Conventions (PEP 8)

13. Exercise: Rename to Redesign

14. Recap



1. Today

By the end of this session you should be able to:

- Explain why a function's **name** is a **design** decision.
- Recognise names that signal a function is doing **too much**.
- Choose names that match what a function **returns** or **does**.
- Spot names that **hide side effects**.
- Name variables consistently and follow Python conventions (PEP 8).



1. Today

By the end of this session you should be able to:

- Explain why a function's **name** is a **design** decision.
- Recognise names that signal a function is doing **too much**.
- Choose names that match what a function **returns** or **does**.
- Spot names that **hide side effects**.
- Name variables consistently and follow Python conventions (PEP 8).

The golden rule: if you can't give a function a clear, simple name, the problem is usually the **design**, not your vocabulary.



Outline

1. Today

2. One Function, One Job

3. Vague Names Hide Vague Purpose

4. Name the What, Not the How

5. Names Must Tell the Truth

6. Grammar Matches the Return

7. Flag Parameters Mean Two Functions

8. Long Names Are a Warning

9. Be Consistent

10. Naming Variables

11. Don't Shadow the Built-ins

12. Python Conventions (PEP 8)

13. Exercise: Rename to Redesign

14. Recap



2. One Function, One Job

- ✓ **DO** a single verb phrase describing **one** action.
- ✗ **DON'T** use names that contain **and**, **or**, **then**, **also**.



2. One Function, One Job

- ✓ **DO** a single verb phrase describing **one** action.
- ✗ **DON'T** use names that contain **and**, **or**, **then**, **also**.

```
# The name confesses: this function does two jobs  
def validate_and_save_student(student): ...
```



2. One Function, One Job

- ✓ **DO** a single verb phrase describing **one** action.
- ✗ **DON'T** use names that contain **and**, **or**, **then**, **also**.

```
# The name confesses: this function does two jobs
def validate_and_save_student(student): ...
```

```
# Split it - and each name becomes simple
def validate_student(student): ...
def save_student(student): ...
```



2. One Function, One Job

- ✓ **DO** a single verb phrase describing **one** action.
- ✗ **DON'T** use names that contain **and**, **or**, **then**, **also**.

```
# The name confesses: this function does two jobs
def validate_and_save_student(student): ...
```

```
# Split it - and each name becomes simple
def validate_student(student): ...
def save_student(student): ...
```

The “and” test: describe the function in one sentence. If you need the word **and**, split it.



Outline

1. Today
2. One Function, One Job
- 3. Vague Names Hide Vague Purpose**
4. Name the What, Not the How
5. Names Must Tell the Truth
6. Grammar Matches the Return
7. Flag Parameters Mean Two Functions
8. Long Names Are a Warning
9. Be Consistent
10. Naming Variables
11. Don't Shadow the Built-ins
12. Python Conventions (PEP 8)
13. Exercise: Rename to Redesign
14. Recap



3. Vague Names Hide Vague Purpose

✗ **DON'T** use filler words: process, handle, manage, do_stuff, data, info, helper.

✓ **DO** say **exactly** what happens.



3. Vague Names Hide Vague Purpose

✗ **DON'T** use filler words: process, handle, manage, do_stuff, data, info, helper.

✓ **DO** say **exactly** what happens.

```
def process(data): ...           # process how? into what?  
def handle_input(x): ...        # handle means... anything
```



3. Vague Names Hide Vague Purpose

✗ **DON'T** use filler words: process, handle, manage, do_stuff, data, info, helper.

✓ **DO** say **exactly** what happens.

```
def process(data): ...           # process how? into what?  
def handle_input(x): ...        # handle means... anything
```

```
def remove_duplicate_emails(emails): ...  
def read_menu_choice(): ...
```



3. Vague Names Hide Vague Purpose

✗ **DON'T** use filler words: process, handle, manage, do_stuff, data, info, helper.

✓ **DO** say **exactly** what happens.

```
def process(data): ...           # process how? into what?  
def handle_input(x): ...        # handle means... anything
```

```
def remove_duplicate_emails(emails): ...  
def read_menu_choice(): ...
```

If the only name you can think of is process, you probably don't yet know what the function is **for**.



Outline

1. Today
2. One Function, One Job
3. Vague Names Hide Vague Purpose
- 4. Name the What, Not the How**
5. Names Must Tell the Truth
6. Grammar Matches the Return
7. Flag Parameters Mean Two Functions
8. Long Names Are a Warning
9. Be Consistent
10. Naming Variables
11. Don't Shadow the Built-ins
12. Python Conventions (PEP 8)
13. Exercise: Rename to Redesign
14. Recap



4. Name the What, Not the How

✓ **DO** name the **intent** (the problem).

✗ **DON'T** leak the **implementation** (the algorithm).



4. Name the What, Not the How



4. Name the What, Not the How

✓ **DO** name the **intent** (the problem).

✗ **DON'T** leak the **implementation** (the algorithm).

```
def bubble_sort_scores(scores): ...      # exposes the algorithm
def multiply_by_point_nine(price): ...   # exposes the arithmetic
```



4. Name the What, Not the How



4. Name the What, Not the How

✓ **DO** name the **intent** (the problem).

✗ **DON'T** leak the **implementation** (the algorithm).

```
def bubble_sort_scores(scores): ...      # exposes the algorithm
def multiply_by_point_nine(price): ...   # exposes the arithmetic
```

```
def rank_scores(scores): ...
def apply_student_discount(price): ...
```



4. Name the What, Not the How



4. Name the What, Not the How

✓ **DO** name the **intent** (the problem).

✗ **DON'T** leak the **implementation** (the algorithm).

```
def bubble_sort_scores(scores): ...      # exposes the algorithm
def multiply_by_point_nine(price): ...   # exposes the arithmetic
```

```
def rank_scores(scores): ...
def apply_student_discount(price): ...
```

If you change **how** it works, the name should **not** have to change.

Sometimes for clarity you may want to include the algorithm in the name, e.g. `rank_scores_quick_sort()`, especially in assessments so that we can check that you



4. Name the What, Not the How

are using the right algorithm. But in general, the name should describe **what** it does, not **how** it does it.



Outline

1. Today
2. One Function, One Job
3. Vague Names Hide Vague Purpose
4. Name the What, Not the How
- 5. Names Must Tell the Truth**
6. Grammar Matches the Return
7. Flag Parameters Mean Two Functions
8. Long Names Are a Warning
9. Be Consistent
10. Naming Variables
11. Don't Shadow the Built-ins
12. Python Conventions (PEP 8)
13. Exercise: Rename to Redesign
14. Recap



5. Names Must Tell the Truth

- ✓ **DO** keep **queries** (return an answer) separate from **commands** (change something).
- ✗ **DON'T** let an innocent-sounding name secretly change state.



5. Names Must Tell the Truth

- ✓ **DO** keep **queries** (return an answer) separate from **commands** (change something).
- ✗ **DON'T** let an innocent-sounding name secretly change state.

```
def get_balance(account):  
    account.balance -= MONTHLY_FEE    # surprise!  
    return account.balance
```



5. Names Must Tell the Truth

- ✓ **DO** keep **queries** (return an answer) separate from **commands** (change something).
- ✗ **DON'T** let an innocent-sounding name secretly change state.

```
def get_balance(account):  
    account.balance -= MONTHLY_FEE    # surprise!  
    return account.balance
```

```
def charge_monthly_fee(account): ...    # command - changes state  
def get_balance(account): ...          # query - safe to call twice
```



5. Names Must Tell the Truth

- ✓ **DO** keep **queries** (return an answer) separate from **commands** (change something).
- ✗ **DON'T** let an innocent-sounding name secretly change state.

```
def get_balance(account):  
    account.balance -= MONTHLY_FEE    # surprise!  
    return account.balance
```

```
def charge_monthly_fee(account): ...    # command - changes state  
def get_balance(account): ...          # query - safe to call twice
```

A function called `get_...` or `calculate_...` should be safe to call **twice**.



Outline

1. Today
2. One Function, One Job
3. Vague Names Hide Vague Purpose
4. Name the What, Not the How
5. Names Must Tell the Truth
- 6. Grammar Matches the Return**
7. Flag Parameters Mean Two Functions
8. Long Names Are a Warning
9. Be Consistent
10. Naming Variables
11. Don't Shadow the Built-ins
12. Python Conventions (PEP 8)
13. Exercise: Rename to Redesign
14. Recap



6. Grammar Matches the Return

Kind of function	Pattern	Example
Performs an action	verb phrase	<code>send_reminder()</code> , <code>print_report()</code>
Returns True/False	<code>is_</code> , <code>has_</code> , <code>can_</code>	<code>is_valid_email()</code> , <code>has_passed()</code>
Computes a value	noun or <code>calculate_</code>	<code>average(marks)</code> , <code>calculate_grade()</code>
Converts	<code>to_ / from_</code>	<code>to_celsius()</code> , <code>from_csv()</code>
May find nothing	<code>find_</code> (may return None)	<code>find_student(student_id)</code>



6. Grammar Matches the Return

Good boolean names make `if` statements read like English:

```
if is_eligible(student):           # reads naturally
    ...

if check_eligibility(student):     # returns a bool? prints? raises?
    ...
```



Outline

1. Today
2. One Function, One Job
3. Vague Names Hide Vague Purpose
4. Name the What, Not the How
5. Names Must Tell the Truth
6. Grammar Matches the Return
- 7. Flag Parameters Mean Two Functions**
8. Long Names Are a Warning
9. Be Consistent
10. Naming Variables
11. Don't Shadow the Built-ins
12. Python Conventions (PEP 8)
13. Exercise: Rename to Redesign
14. Recap



7. Flag Parameters Mean Two Functions

✗ **DON'T** pass True/False to switch behaviour.

✓ **DO** give each behaviour its **own** name.



7. Flag Parameters Mean Two Functions

✗ **DON'T** pass True/False to switch behaviour.

✓ **DO** give each behaviour its **own** name.

```
print_report(students, True)    # True means... what?
```



7. Flag Parameters Mean Two Functions

✗ **DON'T** pass True/False to switch behaviour.

✓ **DO** give each behaviour its **own** name.

```
print_report(students, True)    # True means... what?
```

```
print_summary(students)  
print_full_report(students)
```



7. Flag Parameters Mean Two Functions

✗ **DON'T** pass True/False to switch behaviour.

✓ **DO** give each behaviour its **own** name.

```
print_report(students, True)    # True means... what?
```

```
print_summary(students)  
print_full_report(students)
```

The call site now explains itself - no need to look up the definition.



Outline

1. Today
2. One Function, One Job
3. Vague Names Hide Vague Purpose
4. Name the What, Not the How
5. Names Must Tell the Truth
6. Grammar Matches the Return
7. Flag Parameters Mean Two Functions
- 8. Long Names Are a Warning**
9. Be Consistent
10. Naming Variables
11. Don't Shadow the Built-ins
12. Python Conventions (PEP 8)
13. Exercise: Rename to Redesign
14. Recap



8. Long Names Are a Warning

```
def read_file_parse_rows_and_compute_class_average(path): ...
```



8. Long Names Are a Warning

```
def read_file_parse_rows_and_compute_class_average(path): ...
```

The name is **honest** - and that's the useful part: it shows there are **three** functions waiting to get out.



8. Long Names Are a Warning

```
def read_file_parse_rows_and_compute_class_average(path): ...
```

The name is **honest** - and that's the useful part: it shows there are **three** functions waiting to get out.

```
rows = read_rows(path)
marks = parse_marks(rows)
class_average = average(marks)
```



Outline

1. Today
2. One Function, One Job
3. Vague Names Hide Vague Purpose
4. Name the What, Not the How
5. Names Must Tell the Truth
6. Grammar Matches the Return
7. Flag Parameters Mean Two Functions
8. Long Names Are a Warning
- 9. Be Consistent**
10. Naming Variables
11. Don't Shadow the Built-ins
12. Python Conventions (PEP 8)
13. Exercise: Rename to Redesign
14. Recap



9. Be Consistent

✗ **DON'T** mix synonyms for the same idea:

```
get_student()    fetch_module()    retrieve_grade()
```



9. Be Consistent

✗ **DON'T** mix synonyms for the same idea:

```
get_student()    fetch_module()    retrieve_grade()
```

✓ **DO** choose one word per concept, and use matched pairs:

open / close	add / remove	start / stop	load / save
--------------	--------------	--------------	-------------



Outline

1. Today
2. One Function, One Job
3. Vague Names Hide Vague Purpose
4. Name the What, Not the How
5. Names Must Tell the Truth
6. Grammar Matches the Return
7. Flag Parameters Mean Two Functions
8. Long Names Are a Warning
9. Be Consistent
- 10. Naming Variables**
11. Don't Shadow the Built-ins
12. Python Conventions (PEP 8)
13. Exercise: Rename to Redesign
14. Recap



10. Naming Variables

- **Plural for collections, singular for items:** for student in students:



10. Naming Variables

- **Plural for collections, singular for items:** for student in students:
- **Don't put the type in the name:** names, not name_list or str_names



10. Naming Variables

- **Plural for collections, singular for items:** `for student in students:`
- **Don't put the type in the name:** `names`, not `name_list` or `str_names`
- **Length should match scope:** `i` is fine in a 3-line loop, not across a module



10. Naming Variables

- **Plural for collections, singular for items:** `for student in students:`
- **Don't put the type in the name:** `names`, not `name_list` or `str_names`
- **Length should match scope:** `i` is fine in a 3-line loop, not across a module
- **No cryptic abbreviations:** `total_amount`, not `ttl_amt`



10. Naming Variables

- **Plural for collections, singular for items:** `for student in students:`
- **Don't put the type in the name:** `names`, not `name_list` or `str_names`
- **Length should match scope:** `i` is fine in a 3-line loop, not across a module
- **No cryptic abbreviations:** `total_amount`, not `ttl_amt`
- **Named constants, not magic numbers:** `PASS_MARK = 40`



10. Naming Variables

- **Plural for collections, singular for items:** `for student in students:`
- **Don't put the type in the name:** `names`, not `name_list` or `str_names`
- **Length should match scope:** `i` is fine in a 3-line loop, not across a module
- **No cryptic abbreviations:** `total_amount`, not `ttl_amt`
- **Named constants, not magic numbers:** `PASS_MARK = 40`
- **Avoid negative booleans:** `is_empty`, not `is_not_empty`
(`if not is_not_empty:` is a small act of cruelty)



Outline

1. Today
2. One Function, One Job
3. Vague Names Hide Vague Purpose
4. Name the What, Not the How
5. Names Must Tell the Truth
6. Grammar Matches the Return
7. Flag Parameters Mean Two Functions
8. Long Names Are a Warning
9. Be Consistent
10. Naming Variables
- 11. Don't Shadow the Built-ins**
12. Python Conventions (PEP 8)
13. Exercise: Rename to Redesign
14. Recap



11. Don't Shadow the Built-ins

```
sum = 0
for mark in marks:
    sum += mark

average = sum(marks) / len(marks)  # TypeError!
```



11. Don't Shadow the Built-ins

```
sum = 0
for mark in marks:
    sum += mark

average = sum(marks) / len(marks)  # TypeError!
```

sum is now an int, so sum(marks) fails: **'int' object is not callable**. Avoid list, sum, max, min, id, input, str as variable names.

```
total = 0
for mark in marks:
    total += mark
```



Outline

1. Today
2. One Function, One Job
3. Vague Names Hide Vague Purpose
4. Name the What, Not the How
5. Names Must Tell the Truth
6. Grammar Matches the Return
7. Flag Parameters Mean Two Functions
8. Long Names Are a Warning
9. Be Consistent
10. Naming Variables
11. Don't Shadow the Built-ins
- 12. Python Conventions (PEP 8)**
13. Exercise: Rename to Redesign
14. Recap



12. Python Conventions (PEP 8)

What	Style	Example
Functions and variables	snake_case	calculate_grade, total_marks
Constants	UPPER_CASE	PASS_MARK, MAX_ATTEMPTS
Classes	PascalCase	Student, ShoppingCart
Internal helpers	leading _	_format_row



12. Python Conventions (PEP 8)

What	Style	Example
Functions and variables	snake_case	calculate_grade, total_marks
Constants	UPPER_CASE	PASS_MARK, MAX_ATTEMPTS
Classes	PascalCase	Student, ShoppingCart
Internal helpers	leading _	_format_row

Coming from Java? camelCase works in Python - but PEP 8 will quietly disapprove.



Outline

1. Today
2. One Function, One Job
3. Vague Names Hide Vague Purpose
4. Name the What, Not the How
5. Names Must Tell the Truth
6. Grammar Matches the Return
7. Flag Parameters Mean Two Functions
8. Long Names Are a Warning
9. Be Consistent
10. Naming Variables
11. Don't Shadow the Built-ins
12. Python Conventions (PEP 8)
- 13. Exercise: Rename to Redesign**
14. Recap



13. Exercise: Rename to Redesign

(1) Rename it · (2) split it if the name needs “and” · (3) check for hidden side effects.

```
def do_it(x): ...
def get_and_update_total(order): ...
def handle(scores, flag): ...
def check(s): ...
def calc_avg_and_print(marks): ...
def get_next_ticket():                # hint: look inside
    global ticket_count
    ticket_count += 1
    return ticket_count
```



Outline

1. Today
2. One Function, One Job
3. Vague Names Hide Vague Purpose
4. Name the What, Not the How
5. Names Must Tell the Truth
6. Grammar Matches the Return
7. Flag Parameters Mean Two Functions
8. Long Names Are a Warning
9. Be Consistent
10. Naming Variables
11. Don't Shadow the Built-ins
12. Python Conventions (PEP 8)
13. Exercise: Rename to Redesign
14. Recap



14. Recap

A good name is **evidence** of good design (and renaming often turns into **refactoring**):

- **One job** → one simple verb phrase, no “and”.
- **Clear purpose** → no process, handle, data.
- **Intent, not implementation** → `rank_scores`, not `bubble_sort_scores`.
- **Honest** → `get_` never changes state.
- **Grammar fits the return** → `is_`, `to_`, `find_`, verb phrases.
- **No flag parameters** → two functions, two names.
- **Consistent vocabulary** and **PEP 8**.



Thanks for Watching - Any questions?

